Version of Record: https://www.sciencedirect.com/science/article/pii/S2542660523000197
Manuscript_82914237feccfbbd3097497ae1b6fb33

Internet of Things 22 (2023) 100696

Contents lists available at ScienceDirect

Internet of Things

journal homepage: www.elsevier.com/locate/iot

Research article

Decentralized TSCH scheduling protocols and heterogeneous traffic: Overview and performance evaluation

Mehdi Kherbache [a,*], Otabek Sobirov [a], Moufida Maimour [a], Eric Rondeau [a], Abderrezak Benyahia [b]

[a] CRAN Laboratory, Université de Lorraine, CNRS, UMR 7039, Campus Sciences, BP 70239, F-54000 Nancy, France
[b] LASTIC Laboratory, University of Batna 2, Batna, Algeria

ARTICLE INFO

Keywords:
TSCH
Scheduling
6TiSCH
Simulation
Heterogeneous traffic

ABSTRACT

Advances in the Future Internet and Industry 4.0 are increasing the requirements, especially for Wireless Sensor Networks due to their constrained nature. The IEEE 802.15.4e-TSCH mode has been proposed to meet the needs of such networks and has been largely investigated in the literature to fully exploit its advantages for a wide range of applications, mainly in the industrial automation and process control domains. However, scheduling TSCH transmissions in the time and frequency dimensions is left to implementers. A plethora of TSCH schedulers are proposed in the literature and are developed either in a centralized or a decentralized fashion. Our work aims at giving an overview of the most relevant decentralized TSCH schedulers which we classified into : autonomous, distributed, and Reinforcement Learning based protocols. Then, an evaluation of a selection of these schedulers is carried out in a scenario considering highly heterogeneous traffic, i.e, considering the coexistence of very high and very low data rates in the network. These traffic conditions can be faced in current and future IoT applications, especially surveillance applications. Based on the experiment results, a set of conclusions is drawn for each of the considered schedulers to gain insights into their relevance for applications that need to handle highly heterogeneous traffic.

1. Introduction

Wireless Sensor Networks (WSN) have been proven to have an integral role in the development process of the Internet of Things (IoT) [1,2]. Therefore, advances in WSN can accelerate the realization of the future IoT. Many standards have been issued by several international organizations and alliances to enable the development and deployment of WSNs in several application domains. The IEEE initiated 802.15 Task Group 4e in 2008 to overcome several limitations of IEEE 802.15.4 standard for sharing the communication medium. The new standard referred to as IEEE 802.15.4e [3] defines a number of channel access modes, among which, TSCH (Time Slotted Channel Hopping) [4] designed to meet industrial requirements for timeliness and reliability. The main application domains of TSCH include industrial automation and process control, specifically energy production applications, water management systems, food production plants, pharmaceutical production places, and oil and gas process plants. With the application requirements in mind, one can choose the most appropriate MAC behavior mode of the IEEE 802.15.4e standard. TSCH mode combines time slotting and channel hopping techniques to provide deterministic delay and tries to put some inactive wireless nodes into sleep mode to decrease energy consumption. In particular, when TSCH was combined with IPv6, the range of applications

* Corresponding author.
E-mail addresses: mehdi.kherbache@univ-lorraine.fr (M. Kherbache), otabek.sobirov8@etu.univ-lorraine.fr (O. Sobirov), moufida.maimour@univ-lorraine.fr (M. Maimour), eric.rondeau@univ-lorraine.fr (E. Rondeau), a.benyahia@univ-batna2.dz (A. Benyahia).

https://doi.org/10.1016/j.iot.2023.100696
Received 18 November 2022; Received in revised form 11 January 2023; Accepted 11 January 2023
Available online 14 January 2023
2542-6605/© 2023 Elsevier B.V. All rights reserved.

© 2023 published by Elsevier. This manuscript is made available under the Elsevier user license
https://www.elsevier.com/open-access/userlicense/1.0/

that can be deployed using TSCH increased and this combination was standardized as 6TiSCH by the IETF [5]. However, apart from minimalist schedulers, no standard specifies a complete communication scheduling method for TSCH, which makes it more target-oriented since application requirements can specify how the scheduler should be built.

The literature has witnessed the development of a plethora of TSCH schedulers [6] that can be coarsely classified into centralized or decentralized strategies. This paper aims at giving an overview of the most relevant decentralized TSCH schedulers that we further split into three classes, namely, autonomous, distributed, and RL-based. More importantly, we consider evaluating a selection of representatives among these schedulers, thus, creating their energy consumption and network performance profiles to get some insight into their relevance to applications that have to handle simultaneously highly heterogeneous traffic. We define a scenario that includes two heterogeneous traffic flows qualified as *heavy* or *light* depending on their respective data rates and carry an extensive performance evaluation based on a unified environment under the same network conditions and metrics. Based on the experiment results, a set of conclusions is drawn for each scheduler in an attempt to help choose a scheduler for such type of WSN application. This is motivated by two important observations. First, a wide range of current and future IoT applications need to support heterogeneous flows to fulfill the end-user requirements. In addition to scalar sensors with very low data rates, other sensors with much higher bandwidth requirements such as multimedia ones are more than necessary in surveillance applications [7,8]. In industry 4.0, Visual sensing is of utmost importance to build 3D visualization for the digital twin paradigm [9]. Second, the evaluation of decentralized schedulers has not been investigated, in the literature, under highly heterogeneous traffic conditions. Most of the related work considered low data rate traffic and never highly heterogeneous traffic. In [10], the authors provide a comprehensive analysis of 6TiSCH distributed mode and they carry out a performance evaluation of distributed schedulers. Although extensive performance evaluation is done, autonomous schedulers were not considered and traffic conditions were not heterogeneous. Two autonomous schedulers were assessed in [11] without targeting heterogeneous traffic. This is, also, the case of the work in [12] that additionally considered one distributed scheduler to evaluate with heterogeneous mobility patterns.

To the best of our knowledge, This is the first work that evaluates, in a unified framework, a selection of representative decentralized protocols of different categories (autonomous, distributed, and RL-based) in a scenario that considers heterogeneous traffic conditions with very high and very low data rates simultaneously. The rest of the paper is organized as follows. Section 2 covers the basics of TSCH mode along with an overview of TSCH scheduling. Then, decentralized schedulers are presented and classified into three categories (autonomous, distributed, and RL-based) and the most relevant schedulers of each class are discussed. presents the performance evaluation study, starting with the adopted network model including the considered scenarios. Results are discussed across the considered evaluation metrics and insights about the different evaluated decentralized schedulers are provided. Finally, Section 4 concludes the paper and lists some perspectives.

2. Literature review

2.1. Background on TSCH

TSCH targets mostly Industrial Internet of Things (IIoT) applications by providing a deterministic delay, reliable communication, low power consumption, and multichannel hopping to protect from interference and signal fading. TSCH is built on the combination of two mechanisms: time-slotted access and channel-hopping [4]. The first one allows avoiding collisions between competing nodes while the second one makes possible simultaneous communications between multiple nodes using different channels.

In TSCH, the nodes synchronize to a periodic slotframe consisting of a number of timeslots, which is repeated all along the network life cycle. A slotframe is composed of timeslots where we distinguish two types: *shared* and *dedicated*. In shared slots, communication is based on contention, which implies higher delay and low reliability. On the other hand, dedicated slots are the links that are solely scheduled for peer-to-peer communication between neighbors, thus it provides high reliability. Communication between nodes follows a schedule that is designed to allow nodes to communicate as efficiently as possible based on the application requirements [13]. This schedule can be represented by a matrix whose rows are the different available channels and columns are the timeslots in a slotframe. Each cell of the matrix is a specific link with coordinates (ChannelOffset, SlotOffset). The frequency at which two nodes can communicate in a timeslot is calculated as follows:

$$f = F[(ASN + channelOffset)\%N_{channels}] \tag{1}$$

where ASN is the total number of timeslots that have elapsed since the start of the network service, incremented in each timeslot. Function F is usually defined as the hopping sequence specified in TSCH. The channel hopping mechanism is ensured with Eq. (1) since it can return different frequencies for the same link in different timeslots.

IETF Working Group 6TiSCH is currently working on the standardization of mechanisms that allow the use of TSCH with a low-power IPv6 protocol stack [5]. The goal is to enable the adoption of IPv6 in industrial standards and the convergence of Operation Technology (OT) with Information Technology (IT). However, implementing the IPv6 stack on top of the TSCH MAC layer imposes many challenges. Paper [14] addresses five challenges that arise when implementing 6TiSCH: standards-compliance, timing precision, portability, efficiency and interface with upper layers. To demonstrate the practical feasibility of 6TiSCH, the authors compare their implementation with other state-of-the-art MAC layers (Low-Power Listening (LPL), CSMA) using FIT IoT-LAB [15] testbed with 340 M3 nodes. The results show that synchronization can be achieved with a very low cost (0.3% of radio duty cycle and 0.008% of channel utilization). The results also promise that, with the right scheduling, TSCH can have the highest reliability compared to LLP and CSMA in terms of energy consumption and latency.

Scheduling communication in 6TiSCH has been deemed as an NP-hard problem [16]. In the 6TiSCH standard, no scheduler is defined to leave room for researchers and industries to implement their application-oriented schedulers. The scheduler can be designed based on the application requirements like energy consumption, delay, throughput, packet delivery, etc. TSCH schedulers can be classified into three groups.

Autonomous. This type of schedulers does not exchange any control messages and exploits the existing stored data such as cross-layer routing information, node identifiers or addresses [6]. Orchestra [17] is an example of this type and it builds scheduling locally with the help of the Routing Protocol for Low-Power and Lossy Networks (RPL) [18] control messages. Autonomous schedulers show good network performance in use cases without strict deterministic requirements [6].

Distributed. In this type, nodes negotiate cell allocation with their neighbors to build the schedule. Generally, cell allocation is done based on the type of application. This can provide Quality of Service (QoS) functionality. However, it creates overhead signaling, resulting in more energy consumption. 6TiSCH Minimal Scheduling Function (MSF) is an example of this type of scheduling [19].

Centralized. Centralized schedulers have a main component called Path Computation Element (PCE) which performs all the computation for cell allocation and building the final schedule in the control node. In this kind of network, nodes continuously send report messages to the central node. This kind of scheduler tends to find the optimal communication and differentiate various types of traffic. However, the fact that nodes periodically send report packets to central nodes creates additional network overhead, thus limiting the network bandwidth. Also, as the depth of the network topology (multi-hop) increases, the scheduler goes down in performance. Papers [20,21] explain TSCH integration with Software-Defined Wireless Sensor Network (SDN-WISE) [22] and present an example of a centralized scheduler.

2.2. TSCH scheduling

The authors in [3,6,23–25] extensively studied the TSCH scheduling protocols and listed the advantages and disadvantages of each type of schedule. Additionally, they detailed the challenges of scheduling for TSCH and new approaches being studied in the academic domain.

Paper [3] mainly focuses on IEEE 802.15.4e standard amendment. According to this paper, many factors influence the choice of the scheduling algorithm. For example, as the size of the network increases, centralized scheduling solutions lose performance making them obsolete. The reason is that if a new change occurs in the network topology, the central node has to calculate a new schedule and propagate it to the network. In contrast, distributed and autonomous scheduling can deal with large network sizes. Yet, to some degree, even decentralized schedulers cannot handle the network. In this case, deploying multiple sink nodes benefits communication.

The authors of [23] decided to divide scheduling protocols into main two categories : centralized and distributed. At that time, autonomous scheduling algorithms were being studied to adopt in TSCH. The paper described the limitations and superior features of distributed scheduling protocols. For example, some distributed algorithms rely on shared slots most of the time, which results in non-deterministic performance. Shared slots function as contention-based, that is, nodes compete for the medium. Papers [6,24] concentrate on the same classification and scheduling protocols comparison. However, new approaches have been investigated in [25]. One of the new approaches explained in this study is gaining progressive interest in this research area: building autonomous scheduling protocols based on Reinforcement Learning (RL) [26]. In RL-based scheduling, nodes build scheduling locally without any knowledge of their neighbors' schedules which minimizes energy consumption and bandwidth utilization. Section 2.3.3 illustrates this new approach extensively.

2.3. Decentralized schedulers

In this section, decentralized scheduling strategies for TSCH, which are the main focus of our paper, are further detailed. As depicted in Fig. 1, we mainly classify them into three categories: autonomous, distributed and RL-based, and discuss three well-known schedulers from each class.

2.3.1. Autonomous schedulers

Orchestra [17] is one of the most popular autonomous scheduling algorithms. It builds schedules for each traffic plane (MAC, routing, and application) and maintains them without any overhead communication, mainly using a hash function of the node's ID or its MAC address. It calculates the schedules locally based on the existing topology from the upper layer (exploiting RPL). It has been exclusively developed for TSCH+RPL networks which operate with low-power IPv6 networks. Despite being the first fully autonomous scheduler proposed in the literature, Orchestra is not traffic adaptive since the number of transmission slots does not depend on the traffic volume flowing in the network. Also, collisions may still occur during some of the frames since the cell is allocated only according to a hash of the MAC address. In [17], Orchestra is compared with MSF [19], ContikiMAC [27] and Always-on MAC (CSMA) mode of Contiki-OS [28]. The simulation setup is simple and can evaluate the protocols for general-purpose applications. There are two-directional types of traffic: *sink-to-node* and *node-to-sink*. Packets are generated every 60 s and a real testbed is deployed to test the protocols. The results showed that Orchestra can perform as well as the other protocols while keeping the network synchronized and at low energy consumption. However, the authors did not test the protocols with different types of traffic flows and they only consider a relatively low data rate (1 packet per minute). Currently, Orchestra has three versions: Sender-Based (SB), Receiver-Based (RB) Storing, and RB Non-storing.

Another popular protocol called "Autonomous Link-based Cell Scheduling" (ALICE) is proposed in [29] and widely investigated and studied in [11] with some amendments (multi-channel enabled). ALICE is an autonomous scheduling protocol that deals with resources as links, it allocates cells (timeslots) for a one-directional link between two neighbors. Like any other autonomous scheduler, ALICE takes advantage of the existing RPL routing topology information. Unlike Orchestra, it fully eliminates collisions

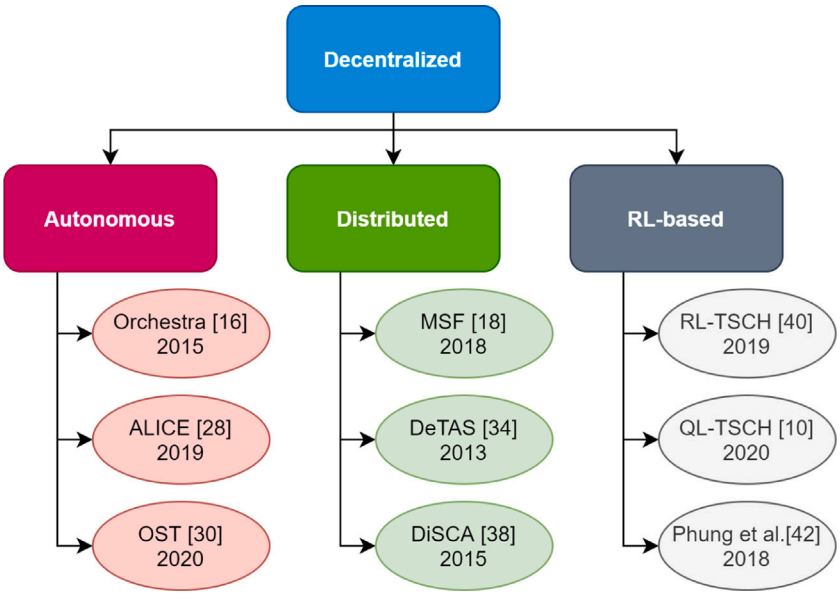


Fig. 1. Decentralized TSCH schedulers classification.

since the hash function used for allocating cells exploits time as an input, represented by Absolute SlotFrame Number, in addition to the link node IDs and the direction of traffic. This way, the hash output is different for each slotframe. Moreover, the upstream and downstream traffic are differentiated in ALICE and they receive separate resources. [11,29] prove that ALICE outperforms Orchestra in all aspects: reliability, throughput, latency, routing stability, and energy consumption. However, it still lacks the adaptive feature since cells are not allocated according to the traffic rate in the network.

To overcome the main limitations of Orchestra and ALICE, i.e., the absence of the traffic adaptive feature, the authors in [30] proposed an autonomous scheduler called "On-Demand TSCH Scheduling with Traffic Awareness algorithm (OST)". This protocol allocates resources based on the traffic load of the nodes. One of its best features is that it can dynamically adjust the frequency of timeslots following the demand in real time. This allows nodes to send their long-queued data in a short time. In this paper, OST is tested against Orchestra and ALICE on a real testbed consisting of 72 nodes (FIT/IoT-LAB). They also generated traffic in two directions (upstream and downstream) but with a low data rate reaching 5 packets/s at maximum for the whole network. The results showed that OST can increase reliability by 60% and energy efficiency by 52% compared to the state-of-the-art protocols (ALICE, Orchestra).

2.3.2. Distributed schedulers

There is a scheduling function that is being standardized by IETF and developed by the 6TiSCH Working Group, called Minimal Scheduling Function (MSF) [19]. This protocol is considered a distributed approach and is designed for generic use cases. It is implemented on top of 6P (*6 top protocol*) and considers three types of cells: minimal, autonomous, and negotiated cells. The minimal cells implement the 6TiSCH minimal configuration that is used for the bootstrap traffic, including a single shared slot for exchanging Enhanced Beacon (EB) packets and RPL (IPv6 Routing Protocol for Low-Power and Lossy Networks) routing packets. The autonomous cells are dedicated to unicast communication. Each node schedules permanently an AutoRxCell (autonomous reception cell) for reception. A node also adds/removes AutoTxCells on demand when there is a frame to be sent and no negotiated cell is allocated for that frame. Finally, the negotiated cells are allocated for a node with its selected parent and they have an adaptive nature. A node can add/delete/relocate negotiated cells with its parent either to match the traffic conditions, to handle switching its parent, or to handle a schedule collision. MSF is regarded as RPL-dependent, which means that an error in RPL can result in network failure. Furthermore, the main drawback of distributed schedulers, and therefore of MSF, is the additional overhead created for negotiating cells allocation between the network nodes. The authors of [31] evaluate the performance of MSF and state that with the right configuration, MSF can keep up with other state-of-the-art autonomous scheduling protocols. In addition, Paper [10] compares MSF with On-The-Fly (OTF) [32] protocol and states that MSF suffers in performance due to the failures in the 6TiSCH stack (mainly RPL configuration).

Decentralized Traffic Aware Scheduling (DeTAS) [33] is one of the first proposed distributed schedulers for TSCH. It uses micro-schedules to build the global schedule where each micro-schedule uses a dedicated set of channels. In fact, each node demands bandwidth from its parent which in turn calculates the number of packets that it will receive from its children and its own local traffic, then transmits this information recursively until it reaches the sink node. This latter starts the cells allocation by scheduling the timeslots to receive the traffic from each child. On the other hand, the allocation of transmission slots in a node is done in such a way as to be synchronized with the reception slot of its parent. In particular, the transmission and reception cells are alternated

along the path to the sink, i.e, if a type of cell is scheduled in an even slot the other type should be scheduled in the next odd slot [34]. In simple words, a packet received during a slot is transmitted to the parent in the next slot. This allows to overcome the hidden terminal problem, avoids buffer overflow and keeps queue utilization as low as possible. Simulation results of DeTAS show better queue management compared to Traffic Aware Scheduling Algorithm (TASA) [35], which is considered as its centralized version, making it resilient to traffic load increment for all nodes in the network. In contrast, its main drawback is that if a packet is lost due to poor radio link quality, all the subsequent slots scheduled on the parent nodes to forward this packet are wasted. In addition, the same limitation as its centralized version (TASA) is present, i.e., when the network topology or traffic changes, the schedule procedure has to start over to still match the latency requirements. Moreover, collisions may occur if other links exist in addition to the convergecast links [36].

The authors in [37] proposed a distributed interference and traffic-aware scheduler for TSCH called DiSCA (Distributed Scheduling for Convergecast in multichannel WSNs Algorithm). It tries to minimize the number of timeslots needed to transmit all the packets to the sink. In fact, the algorithm operates iteratively by allocating at the step i the ith transmission of each node. The priority in terms of scheduling is given to the node which has the highest number of packets to transmit. Each iteration results in a micro-schedule that can overlap to reduce the total number of slots. Moreover, a transmitting node notifies all its conflicting nodes upon picking the first available pair of timeslot and channel offset. DiSCA takes into account the case where the sink is equipped with multiple radio interfaces and considers different traffic loads generated by the nodes : experiments have been conducted considering that each node generates a number of packets randomly drawn in $[1, 5]$ per slotframe. According to the simulation results in the original paper, DiSCA outperforms Wave [36] which was proposed previously by the same authors, in terms of the number of required slots in different configuration setups. Also, it gives a close performance to the optimal schedule with reduced control messages. In contrast, the drawback of such an approach resides in notifying the reservations to the interfering nodes which is a complicated task in practice. The set of conflicting nodes has to be precisely identified and estimating the interfering nodes is still an open challenge. Furthermore, no mechanism tries further to detect and solve the inconsistencies (collisions).

2.3.3. RL-based schedulers

RL is a sub-field of machine learning domain [26]. An RL-based model is a goal-driven process where there is an agent with several actions to take in order to reach its final goal. The agent interacts with its environment using available actions to learn and reach the target. After performing an action, the agents earn a reward value and based on this reward, it either explores other actions (exploration) or takes the action with the highest gained reward (exploitation). Our survey paper [38] covers most of the modern RL-based scheduling algorithms for single-channel and multi-channel TDMA.

One of the first proposed RL-based schedulers for TSCH is called RL-TSCH [39]. RL-TSCH exploits only one channel of the medium and uses RL to minimize collisions between transmitting nodes. It is designed to take into account the number of packets in the transmission buffer along with the remaining energy level in every node. RL-TSCH decides to turn on/off the node's radio at the beginning of each timeslot according to the current and the previous state of the node. Thus, the learning agent determines the number of active and non-active timeslots at the beginning of each slotframe. During the active timeslots, it behaves as in the Minimal Scheduling Algorithm (MSA) of TSCH. Otherwise, during the non-active timeslots, it turns off its radio. The algorithm applies Q-learning [40] with an action space of choosing the number of active slots. The reward function is designed to minimize energy consumption and ensure high throughput and reliability. The experiment results showed that RL-TSCH reduces considerably the energy consumption compared to MSA. In addition, the reward function is designed to make it traffic adaptive but on the other hand, the algorithm has the drawback of not being well suited for applications with strict real-time requirements.

A more promising single channel RL scheduler is QL-TSCH [41] protocol which was built on the basis of Q-Learning. It allows contention but tries to minimize collisions. In this protocol specification, there are two slotframes: broadcast and unicast. The broadcast slotframe has 7 timeslots length, and only the first slot is allocated as shared transmission(Tx)/reception(Rx) (during the other slots, a node turns off its radio). On the other hand, the unicast slotframe has different sizes (9,15,25) and it is for application data communication. Only one slot is designated as a Tx cell and all the other slots are Rx cells.

A node acts as a QL agent to learn its environment and neighbors to find which slot it should allocate as Tx. QL-TSCH addresses the non-stationarity problem of the multi-agent system that may interrupt the convergence of the algorithm by including an Action Peeking mechanism (AP). A node listens to the medium during Rx slots and it records this information in the Action Peeking Table (APT), having the size of the unicast slotframe. In simple words, the APT records the percentage of usage of each slot. Therefore, the protocol chooses the minimum value (the less used slot) in the APT table and selects it as a Tx slot.

When a node initiates communication on the Tx slot, it gets a reward based on the status of transmission: 0 for a successful transmission or -1 for a failed transmission. This $0/-1$ reward design has been proved to perform best by the authors of QL-TSCH after testing various settings. The node builds a Q-table that has the size of the unicast slotframe. And after receiving the reward value, the node updates the corresponding Q-value in the Q-table.

After every cycle of the unicast slotframe, the Q-table and the APT table will be updated, and a new Tx slot will be scheduled. QL-TSCH operates with an exploration/exploitation mechanism. In the former case, a node updates its Q-table and APT after transmission on the Tx slot but chooses the next Tx slot based on APT table values. This means the node is exploring its environment by trying different timeslots and updating its Q-table. In the latter case, the node chooses the Tx slot based on its Q-table value, that is, a timeslot with the highest Q-value indicating that the node earned most of the rewards from this timeslot. QL-TSCH has a policy update function that urges nodes to choose exploration at the beginning and exploitation as time passes.

Experiment results of QL-TSCH in a testbed of 99 nodes showed that it outperforms Orchestra in terms of Packet Delivery Ratio (PDR) and gives lower end-to-end delay. However, energy consumption has not been evaluated which raises the question of its effectiveness in real industrial energy-constrained scenarios.

Table 1
Our naming conventions for the Evaluated protocols.

Reference name	Protocol explanation
Orchestra	Orchestra protocol with Link-based rule,
OST	OST protocol with Unicast slot frame adaptation and default configuration
ALICE_rx_sf7	ALICE protocol with channel hopping and the unicast slot frame size is 7
ALICE_sf7	ALICE protocol without channel hopping and the unicast slot frame size is 7
ALICE_rx_sf19	ALICE protocol with channel hopping and the unicast slot frame size is 19
ALICE_sf19	ALICE protocol without channel hopping and the unicast slot frame size is 19
MSF	Minimal Scheduling Function for 6TiSCH protocol (version 0.2)
QL-TSCH-5	QL-TSCH protocol with a unicast slotframe size of 5
QL-TSCH-9	QL-TSCH protocol with a unicast slotframe size of 9
QL-TSCH-15	QL-TSCH protocol with a unicast slotframe size of 15

In the category of multi-channel RL-based schedulers, a scheduler for TSCH networks supporting multiple Quality of Service (QoS) objectives has been proposed in [42]. It is based on a trial-and-error process where each node acts as an autonomous agent learning from its environment based on the feedback it receives from it. Also, nodes cooperate to attain the objective of low energy consumption.

The authors considered two RPL instances where the first one is dedicated to delay-sensitive data and the second to regular data. The action space consists of (i) the node listens for data from child nodes, (ii) forwards data to the parent of the delay-sensitive instance or (iii) forwards to the parent of the regular instance. The reward function is designed as a combination of rewards for the low latency objective and rewards for the objective of reliability and energy. Based on the received reward signal, the action to be executed in the next slotframe will be the same as in the current slotframe if the reward is equal to 1 or another action will be selected if the reward is equal to 0. The probabilities of successfully performing each action in each slot are recorded. Then, these probabilities are exploited to choose the best actions for each node during the slotframe, this is called the allocation phase.

This scheduler has been evaluated against Orchestra in a Matlab simulation, considering two RPL instances where: one is dedicated to delay-sensitive data (bounded latency set at 2 timeslots) and the other is for regular data. Results showed that the learning-based scheduler ensures one order of magnitude lower data delivery latency compared to Orchestra while consuming the same amount of energy. However, the drawback of this scheduling scheme is that the algorithm's convergence is not guaranteed.

3. Performance evaluation

The main focus of our evaluation is to assess the performance of different categories of TSCH schedulers under highly heterogeneous traffic. To do so, we selected representative protocols from each category of decentralized schedulers according to our classification provided in Section 2.3. In the category of autonomous schedulers, we considered the three protocols reviewed in Section 2.3.1, namely, Orchestra, ALICE, and OST. This is motivated by the fact that while these schedulers do not add network overhead, they proved to achieve very good performance in various scenarios [11,17,29,30]. Among the RL-based schedulers, we selected and implemented QL-TSCH as it outperforms Orchestra as reported in [41]. Distributed schedulers are based on negotiation between neighbors which incurs additional network overhead. This is why we limited our experiments to one distributed protocol and chose MSF as it is a potential standard scheduler for the 6TiSCH protocol stack designed for generic use cases. Our present work is the opportunity to assess the ability of MSF and the other selected protocols to handle the co-existence of highly heterogeneous traffic. The considered protocols along with the adopted naming conventions with respect to the different settings are listed in Table 1.

In order to assess the performance of each protocol in the presence of highly heterogeneous traffic, one needs to carry out an evaluation study using a unified environment with the same network model and metrics. Our experiments have been conducted using COOJA simulator [43] provided by Contiki-NG [44], an operating system for the IoT that implements a low-power IPv6 networking stack. We consider a square field (Fig. 2) where the Sink (node 1) is located in the center and 64 sensor nodes are deployed in a 8×8 grid topology with a distance of 45 meters between two consecutive nodes in the grid. The sensor nodes are equipped with an IEEE 802.15.4 radio and the propagation model is the well-known Unit Disk Graph Medium (UDGM). We chose to use Cooja motes which are generic sensor nodes because they are the only ones with which all the selected schedulers are actually able to function successfully. To estimate the energy footprint of the network, we adopted an energy model of the Sky Motes [45] and made use of the Energest module of Contiki-ng invoked periodically every 10 s.

Extensive simulations have been conducted in two scenarios. In the first scenario, nodes do not exchange any application data packets. The aim is to check how the scheduling protocols behave in the absence of data flows to chiefly analyze the network

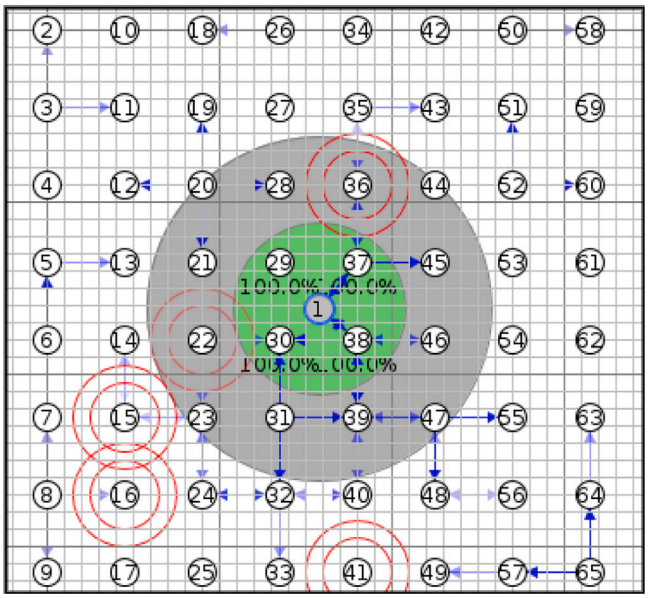


Fig. 2. Network Topology.

convergence with the different protocols and how a protocol consumes energy in an idle state. In the second scenario, we consider that sensor nodes are in charge of reporting information from the area of interest to the Sink (RPL root node) following a delivery model known as *convergecast* or many-to-one communication. Some of these sensors (nodes 2, 9, 58, and 65) are ideally placed at strategic positions (the four corners of the grid). They are responsible for gathering and transmitting data with a high reporting rate to generate the *heavy traffic* in our experiments. Each heavy flow consists of the transmission of two packets per second (2 pps) with a payload of 50 bytes resulting in a transmission rate of 800 bps (bits per second). Each of the remaining 60 nodes transmits with a rate of only one packet per minute with a payload of 10 bytes resulting in a transmission rate of 1.33 bps. Similarly, we use the terms *light traffic* to designate these latter flows. The sink does not generate any packets since its role is to receive packets. This setting with multiple flows, some of which exhibiting high data rates, will likely create bottlenecks close to the sink due to the convergecast scheme. The results will show how the chosen TSCH schedulers manage the situation. This latter scenario is to actually evaluate the protocols in normal conditions with respect to our objectives, i.e., to assess their performance under highly heterogeneous traffic.

Each simulation lasts 30 min after the network convergence time which is the time required for all the nodes to join the network successfully (network steady state). Different settings of the chosen schedulers have been considered, resulting in a total of 10 protocol configurations. We run each protocol configuration 10 times with a different random seed for each scenario to increase the confidence interval of our simulations, mounting up to $10 \times 10 \times 2 = 200$ simulations in total; 100 with no application data and 100 with application data generation. The main simulation parameters are listed in Table 2.

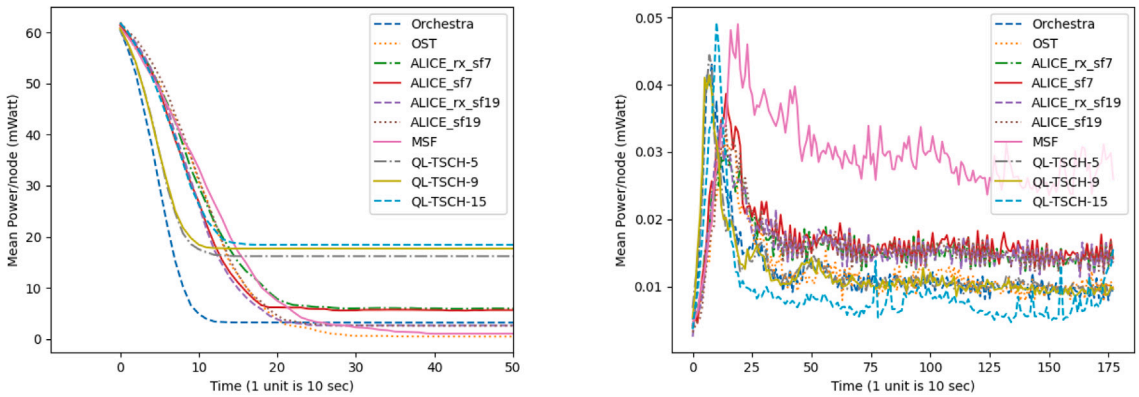
3.1. Network convergence time

We report hereafter on the obtained results when the nodes do not exchange any application-level data packets to analyze the network convergence with the chosen schedulers. This is done based on the study of the evolution of power expenditure over time which gives indirectly insight into how long it takes for the network to converge. The convergence time is the time required for the network to be in a steady state where all the nodes have joined the network and all configurations (MAC and routing level) have been successfully established.

Fig. 3(a) depicts the mean power consumption per node over time. This gives insights into how the network consumes power with each protocol to converge as a whole. We observe that Orchestra clearly builds the network faster than the other protocols, resulting in lower power consumption over a shorter period of time (around 100 seconds). Similarly, QL-TSCH protocol with a slotframe size of 5 and 9 have approximately the same result. However, after the network convergence, the power consumption of this protocol is the highest (around 20 *mWatt*). This is mostly due to its action peeking mechanism which keeps a node listening to communications between its neighbors, thus increasing power usage. QL-TSCH can converge faster because it uses a broadcast slotframe with one shared slot having a higher priority than the unicast slotframe. Additionally, a short slotframe size enables faster convergence, for instance, QL-TSCH-5 and QL-TSCH-9 converge after 100 seconds while QL-TSCH-15 converge only after 150 seconds. We can note that OST and MSF are the schedulers that take the longest time to converge (about 300 seconds) but they also provide the lowest (almost zero) power consumption after convergence. This confirms that OST provides 'just enough' slots for each link without any additional control overhead, which minimizes energy consumption as the figure shows.

Table 2
Simulation parameters.

Parameter	Value
Simulation time	30 min (after network convergence)
Mote type	COOJA mote
Protocol stack	6TiSCH
Transport protocol	User Datagram Protocol (UDP)
Routing protocol	RPL Classic
Network topology	Grid 8 × 8
Number of nodes	65
Distance between nodes	45 meters
Propagation model	Unit Disk Graph Medium (UDGM)
Interference range	100 meters
Tx range	50 meters
Heavy traffic	4 nodes, each transmitting at 800 bps
Light traffic	60 nodes, each transmitting at 1.33 bps
Timeslot duration	10 ms
Maximum re-transmissions	3
Packet queue size	16
Power Radio Rx	65.4 mw
Power Radio Tx	58.5 mw
Power CPU	7.2 mw
Power Low Power Mode (LPM)	3.6 mw



(a) Overall power consumption

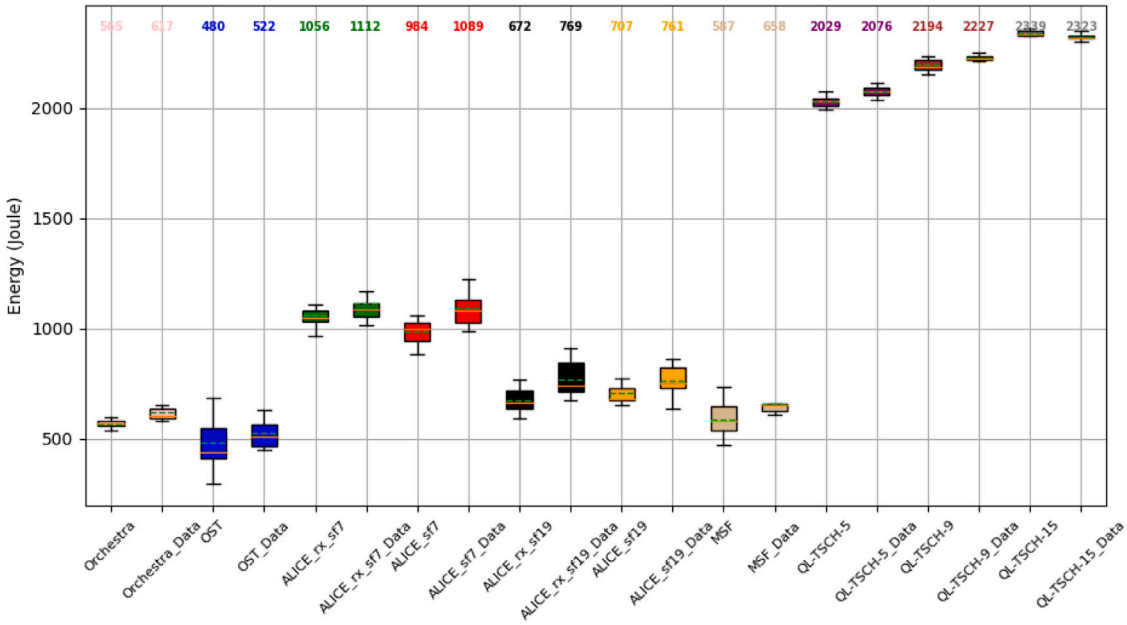
(b) Transmission power consumption

Fig. 3. Mean power consumption per node over time.

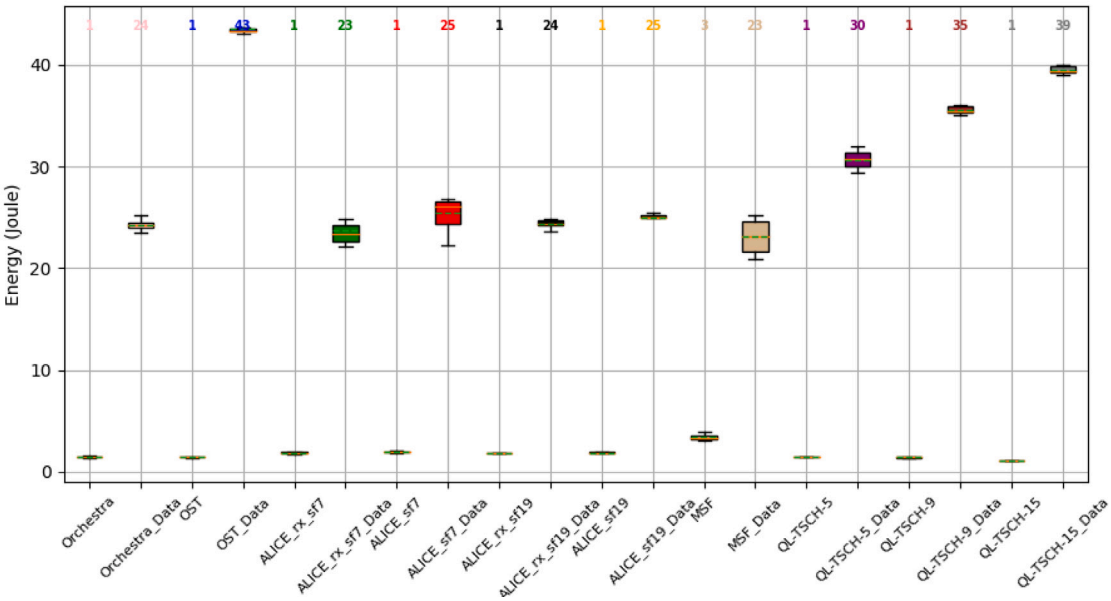
Based on our experiments, The overall power averaged on a 10-second period is mainly driven by the radio reception mode which accounts for about 95% of the total consumption. As a result, the latter follows the same pattern as the overall power. Fig. 3(b) illustrates the mean transmission power usage per node which, as can be seen, is much lower than the overall and reception mode consumption. This can provide more insight into the actual exchanges that take place during the convergence phase. Initially, in the [0, 250]-second interval, we observe a high transmission power consumption for all the protocols. This is due to the networking convergence or bootstrapping process. In order for the network to converge, various control packets are exchanged between nodes. For instance, at the TSCH level, upon receiving an Enhanced Beacon (EB) message, considered as a request to join the TSCH network, a node joins this latter and broadcasts the message to its unknown neighbors to invite them to join the network in turn. Moreover, other control messages are exchanged at the routing level to build the DODAG (Destination Oriented Direction Acyclic Graph) of RPL. Once the network has converged, we note that MSF consumes more power for transmission than other protocols. This is due to its distributed nature where negotiations between nodes are performed for cells allocation using *6top* protocol (6P), resulting in additional overhead and thus more power consumption. This is unlike autonomous and RL-based schedulers which make the scheduling decisions based on local information without additional communication overhead for nodes cooperation. Another interesting fact is that QL-TSCH protocol with a slotframe size of 15 has the lowest power usage. This is explained by the fact that the transmission slot for control messages comes less frequently due to a longer unicast slotframe.

3.2. Energy consumption

In this Section, we are interested in the total amount of energy consumed by the network during the whole simulation time. We consider the two scenarios defined before. In the first scenario, there is no application data exchange and protocols run to keep



(a) Total energy



(b) Transmission energy

Fig. 4. Network energy consumption in the absence or the presence of application data exchange ("_Data" refers to the latter scenario).

the network up and functioning, while in the second scenario, the data collection application runs on every node and the sink (the sink collects data from source nodes). The obtained results are presented using box plots to illustrate their distribution. The lower and upper whiskers represent the minimum and maximum values, respectively. The box represents values from the first to the third quartile including the median value. The average values are highlighted above each box.

Fig. 4(a) compares the energy consumption of the protocols in both scenarios. From the figure, it can be seen that there is not any big difference between the two scenarios for most of the protocols. OST proves to have the lowest energy consumption for both scenarios, although the distribution is more compact when exchanging application data (*OST_Data*). It is followed by Orchestra and MSF with a small increase of 70–90 Joules. Yet, we should keep in mind that energy consumption is not the only decisive factor when choosing a scheduling protocol for WSN. The idea is to find the trade-off between all the metrics according to the application requirements. QL-TSCH in both scenarios performed poorly due to its Action Peeking mechanism which keeps the nodes listening for communications between neighboring nodes [41]. The ALICE protocol achieves lower energy usage with larger

slotframe sizes. Furthermore, it can be concluded that whether or not channel hopping is enabled does not significantly affect the energy consumption of ALICE.

In Fig. 4(b), a comparison is given for the transmission power of the chosen protocols under the two considered scenarios. As expected, the protocols spend more energy when nodes exchange application data. All the figures are self-explanatory, except for OST whose energy usage for transmission is surprisingly high, with an average of 43 Joules. The explanation might depend on the network performance metrics, e.g., the protocol might have spent more power due to packet re-transmissions (Section 3.3). But this does not affect the overall energy efficiency of OST which performed best in our case. Additionally, as is the case for the total energy consumption, QL-TSCH consumes more energy for transmission compared to the other schedulers (with the exception of OST). This is because it uses only a single channel for transmission which increases the probability of collisions, although the Action Peeking mechanism reduces this latter but it does not eliminate it. When a collision occurs, a re-transmission is planned for the packet that did not receive acknowledgment affirming its successful reception, thus increasing the transmission power usage.

Another intriguing point in this figure is that for all the protocols in the scenario without application data, their transmission energy consumption equals 1 *Joule* except for MSF is almost three times higher (3 *Joules*). This is due to the same reasons stated in Section 3.1, i.e., the additional overhead for negotiating cells allocation since it is a distributed protocol.

3.3. Network performance

This section sheds light on the comparative performance evaluation of the selected protocols in the second scenario, i.e., with heterogeneous traffic being generated to be transmitted to the Sink. We consider the following network performance metrics :

Packet delivery ratio (PDR). The ratio of the number of packets received by the sink to the number of packets sent by the source nodes.

End-to-end delay. The time that elapses between the transmission of a packet by a source node and its reception by the sink.

Network throughput. The amount of data successfully received, in a time unit, by the sink transmitted by the different source nodes, expressed in *bps*.

Fig. 5 depicts delay and PDR achieved by each protocol overall and separately for heavy and light traffic flows described in Table 2. When both metrics are considered, ALICE with slotframe size 7 outperforms the other protocols, either with channel hopping enabled or not. However, the enabling of channel hopping in ALICE increases the performance as can be noticed in the figure. ALICE with channel hopping enabled achieves better performance in both slotframe sizes than with channel hopping disabled. Even though ALICE with a slotframe size of 7 has higher power usage, it performs better than ALICE with larger slotframes.

Orchestra has a very high PDR level but it provides a relatively higher average delay compared to ALICE_rx_sf7 or QL_TSCH_5. Overall, we can say that all the protocols present reasonable performance except for QL-TSCH of slotframe sizes 9 and 15. However, it is thought-provoking that the QL-TSCH with a smaller slotframe size can outperform the other protocols in terms of delay. This kind of protocol can be useful in time-sensitive wireless sensor networks without strict reliability requirements, yet nodes should be provided with more energy storage.

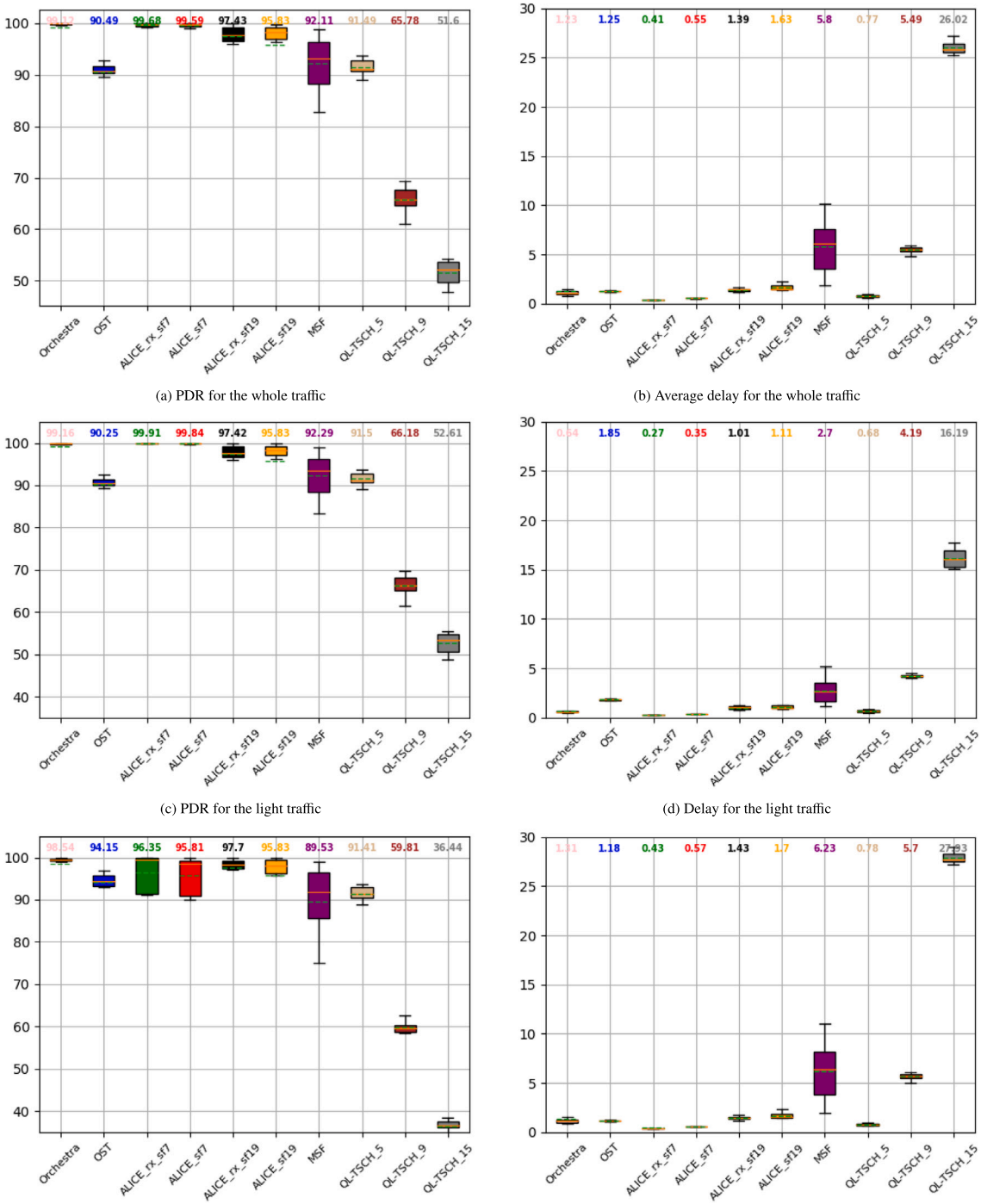
On the other hand, OST suffers in performance to some extent although it has lower energy consumption. This is mainly true for light traffic as it should be mentioned that OST handles heavy traffic better than the light one in terms of Delay and PDR. Fig. 7 plots the network throughput over time for heavy and light traffics and confirms the previous statement as can be noticed that OST (represented with a dotted orange line) gives higher throughput for heavy traffic while with light traffic, the achieved throughput is medium compared to the other protocols.

Another most distinctive box-plot in Figs. 5(e) and 5(f) belongs to MSF that, putting aside QL_TSCH_9 and QL_TSCH_15, handles very poorly the heavy traffic. Also, it exhibits a wide variation compared to the other protocols, this can be explained by its additional overhead to negotiate cells allocation and adapt traffic load. This incurs more congestion in the network, thus higher delays and packet drops occur in some simulations. This allows concluding that MSF does not handle very well heterogeneous traffic despite its energy efficiency as discussed in Section 3.2.

The average throughput of each protocol is depicted in Fig. 6. It is obvious that Orchestra shows the best performance, which proves that Orchestra is an appropriate protocol for applications requiring very high reliability and acceptable delay. Fig. 7 depicts the achieved throughput by the different protocols over time for the light and heavy traffic. The blue lines show the expected throughput for each traffic flow. The expected throughput is calculated theoretically to show the achievable data reception rate. For heavy traffic: 4 nodes send 800 *bps*, which results in $4 * 800 = 3200$ *bps* for the whole network. As for the light traffic: 60 nodes send $10 * 8/60$ *bps* each, resulting in 80 *bps* for the whole network.

Fig. 7(a) showcases throughput levels for the light traffic. It is clear that QL-TSCH with larger slotframe sizes could not fully handle light traffic. Furthermore, there is no significant difference between other protocols, showing that they all performed similarly in terms of throughput for light traffic evolution over time. Note that some levels are above the expected throughput because packets may be received with a variable jitter. Packets may be queued in the buffer and delivered in bursts, not to mention the effect of retransmissions that can instantaneously increase the transmission rate.

Fig. 7(b) depicts the throughput for the heavy traffic for each protocol over time. Once again, QL-TSCH with slotframe sizes of 9 and 15 performed worse in contrast to the other protocols. In addition to low throughput, they have a lot of fluctuations, which means that higher data rates impact QL-TSCH ability to keep stable. As for MSF, it takes a longer time for network convergence and even when it does, the achievable throughput does not get near the expected throughput. Other protocols have a good performance.



(a) PDR for the whole traffic

(b) Average delay for the whole traffic

(c) PDR for the light traffic

(d) Delay for the light traffic

(e) PDR for the heavy traffic

(f) Average delay for the heavy traffic

Fig. 5. PDR (%) and average delay (s) results.

Interestingly, Orchestra maintained a higher throughput level during the whole network operation, which proves its previously stated feature of being highly reliable. ALICE of slotframe length 7 (with or without channel hopping, red and green lines, respectively) kept high throughput levels for heavy traffic until it drops starting approximately from the 1150 second. This helps to figure out when packet drops happened since its PDR average value is around 95% in the case of heavy traffic, as shown in Fig. 5.

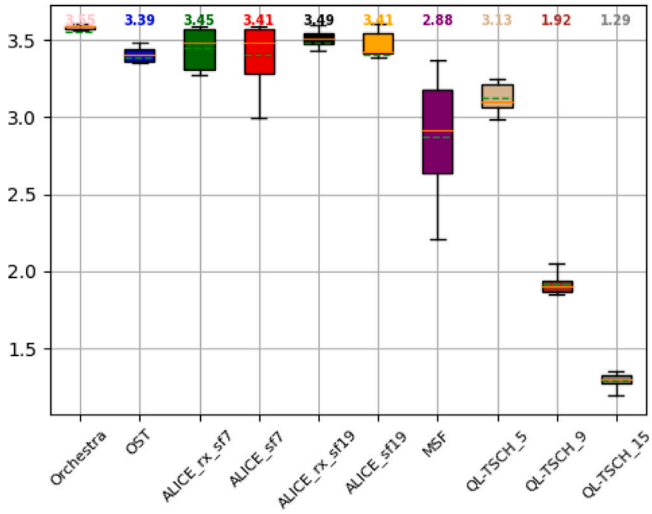


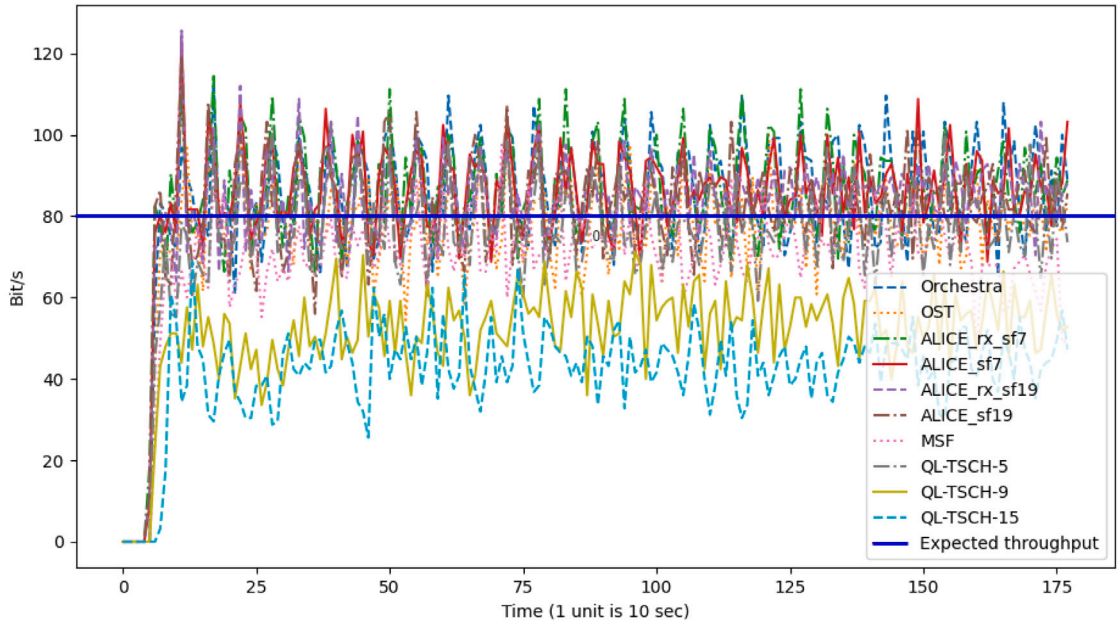
Fig. 6. Average throughput of the network in Kbps.

3.4. Results summary

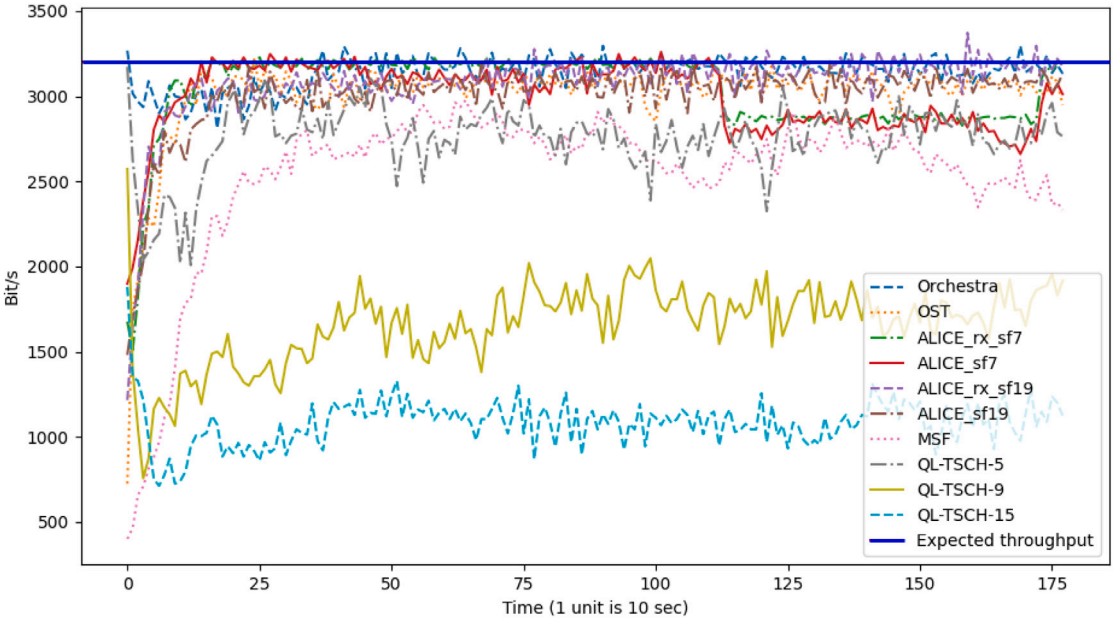
Based on the results of the experiments, a set of general conclusions is drawn for each scheduler :

- QL-TSCH protocol performs well with smaller slotframe sizes. In fact, it has two slotframes : broadcast and unicast. The ratio of unicast/broadcast slotframe size determines performance. If this ratio is less than 1, QL-TSCH will have frequent Tx slots, hence increasing the throughput. QL-TSCH with a slotframe size of 5 has never been investigated. We analyzed this and concluded that QL-TSCH-5 performs better than QL-TSCH-9 and QL-TSCH-15. However, the energy usage of this protocol is overwhelmingly due to its Action Peeking mechanism. We would recommend improving or replacing this mechanism by keeping in mind the goal of making a trade-off between reducing collisions and keeping low energy consumption. As a first idea, incorporating a simple negotiation mechanism between neighboring nodes to help choose the least-used timeslot. Although this would generate additional network overhead but it would surely consume less energy than keeping nodes listening for communications during the entire slotframe, as is the case with the action peeking mechanism. Moreover, enabling the use of multiple channels with QL-TSCH would further increase its performance and allow it to fully exploit the potential of TSCH.
- The performance of MSF is not the worst in any metrics. This protocol is currently being standardized as a TSCH scheduling protocol by the 6TiSCH Working Group [19]. MSF will likely keep its reputation by performing well for any general type of WSN application.
- ALICE protocol showed good performance in most of the metrics, yet its power usage is higher than Orchestra and OST. Like QL-TSCH, ALICE can handle heterogeneous traffic better with smaller slotframe sizes. We tested two different versions of ALICE protocol : the original version [29] and the new improved version (node-based version by [11]). The node-based version has a slightly lower power consumption compared to the original version. Also, the new version uses multi-channel hopping, which makes the protocol robust to interference and multipath fading.
- The lowest energy consumption was recorded by OST protocol, followed by Orchestra. Even under high data rates, OST kept its low energy consumption while maintaining an acceptable packet delivery ratio. PDR is not the highest metric for OST, yet it has a much lower latency compared to other protocols. OST has the potential to be used in extremely energy-constrained networks where a lower PDR is reasonable.
- Finally, Orchestra overcame most of the protocols in terms of energy consumption and network performance. It presented the shortest convergence time, meaning that it can adapt to sudden changes in the network topology faster. Orchestra's network performance cannot be dismissed since it provides high reliability and a reasonable delay. The energy consumption of Orchestra is not the lowest value in our list, yet it is almost the same as OST which is the lowest in our records.

On the other hand, the application requirements are to be considered in order to choose the best-fit scheduler, as previously stated. One can adopt a multi-criteria analysis of the protocols to help in deciding which protocol is suited for a specific application. Hereafter, we give an illustrative example where we consider a simple method in which a linear utility function is defined for each criterion, each of which corresponds to one evaluation metric. The utility function permits normalizing the metrics values in the range $0 \ldots 1$ where the value 1 is associated with the best value of the criterion. For instance, in the utility function defined for energy consumption, the minimum value should be the value associated with this criterion. On the other hand, a maximum value is to be considered when it comes to reliability (estimated via the PDR metric). Table 3 summarizes the average values of the evaluated protocols for the four criteria (metrics) : Energy consumption, Delay, PDR, and Throughput. Since QL_TSCH_9 and QL_TSCH_15



(a) Light traffic



(b) Heavy traffic

Fig. 7. Network throughput over time.

performed very poorly, we eliminate them from our choices for statistical purposes. Let $E(x)$, $D(x)$, $P(x)$, and $T(x)$ be the utility function for, respectively, energy consumption, end-to-end delay, packet delay ratio, and throughput. Based on the minimum and maximum values achieved, we obtain the following expressions :

$$E(x) = (-x + 2076)/1554$$
$$D(x) = (-x + 5.8)/5.39$$
$$P(x) = (x - 90.49)/9.19$$
$$T(x) = (x - 2.88)/0.77$$

Table 3
Multi-criteria average values of the different protocols.

Criterion	Orchestra	OST	ALICE rx sf7	ALICE sf7	ALICE rx sf19	ALICE sf19	MSF	QL TSCH 5
Energy (Joules)	617	522	1112	1089	769	761	658	2076
Delay (seconds)	1.23	1.25	0.41	0.55	1.39	1.63	5.8	0.77
PDR (%)	99.12	90.49	99.68	99.59	97.43	95.83	92.11	91.49
Throughput (Kbps)	3.65	3.39	3.45	3.41	3.49	3.41	2.88	3.13

Table 4
5-Level scale criterion preferences.

Criterion	Surveillance Application		Industrial application	
	Score	*Weight (%)*	*Score*	*Weight (%)*
Energy	5	0.42	1	0.07
Delay	1	0.08	5	0.36
PDR	2	0.17	5	0.36
Throughput	4	0.33	3	0.21

Table 5
Aggregation values.

Protocol	Surveillance application (A_1)	Industrial application (A_2)
Orchestra	**0.953**	**0.920**
OST	0.705	0.511
ALICE rx sf7	0.738	0.908
ALICE sf7	0.742	0.895
ALICE rx sf19	0.807	0.789
ALICE sf19	0.745	0.690
MSF	0.413	0.128
QL TSCH 5	0.199	0.442

After defining the utility functions, a preference (score) is set to each criterion on a 5-level scale. The preferences are to be specified based on the application requirements. In this example, we consider an environment surveillance application with multimedia traffic and an industrial application. The former requires mainly high throughput and energy efficiency, while the latter has strict requirements for real-time and reliability. Table 4 defines the weighting of each criterion according to the target application.

Finally, an aggregation function is used in order to choose which protocol is best suited for a specific application. It assigns a score to each solution by accumulating the utility functions of the predefined criteria, multiplied by their respective weights. Let $w_1, w_2, \dots, w_n$ be the weights of criterion $1, 2, \dots, n$ and $F_1, F_2, \dots, F_n$ their respective utility functions. Also, we can characterize a solution (or a protocol in our case) with a vector, $X = (x_1, x_2, \dots, x_n)$. The aggregation function is defined as the weighted average of the utility functions: $A(X) = \sum_i w_i \ F_i(x_i)$. This way, the aggregation function for the environment surveillance and the industrial application respectively, can be given by :

$$A_1(X) = 0.42 \ E(x_1) + 0.08 \ D(x_2) + 0.17 \ P(x_3) + 0.33 \ T(x_4)$$
$$A_2(X) = 0.07 \ E(x_1) + 0.36 \ D(x_2) + 0.36 \ P(x_3) + 0.21 \ T(x_4)$$

Table 5 contains the aggregation function values of each protocol in the context of environment surveillance (aggregation function A_1) and the industrial one (aggregation function A_2). From the table, we can conclude that Orchestra is the best-suited protocol for the environment surveillance application and also for the industrial application because it has the highest aggregation function value in both cases, 0.953 for the former and 0.920 for the latter application.

4. Conclusion

TSCH protocol provides deterministic and reliable communication by the means of time-slotted access and channel hopping. Yet, it has an open research field for the scheduling scheme. There are two main approaches for scheduling : centralized and decentralized. Decentralized schedulers have the potential to provide better performance for WSNs because they adapt well to network changes and less networking overhead is generated compared to centralized schedulers. In this work, an overview of decentralized schedulers has been provided, including classification into three main categories : autonomous, distributed, and RL-based. In the literature, the evaluation of such schedulers has not been investigated under highly heterogeneous traffic conditions. Hence, we defined a scenario including two heterogeneous traffic flows (heavy and light) to assess a selection of the most representative decentralized schedulers based on a unified environment under the same network conditions and metrics.

To conclude, highly heterogeneous traffic conditions are challenging because various data flows with different rates and requirements coexist in the same network. Hence, more effort must be made to ensure efficient communication. As a future work,

one may consider scheduling on a per-flow basis depending on the application needs. For instance, light traffic, issued from a scalar sensor, typically requires timeliness and reliability while heavy traffic, from a visual sensor, needs more bandwidth and less reliability. More scenarios with other metrics especially in a real-world publicly available testbed such as IoT-LAB [15], are to be designed for other use cases to observe and experience real-world WSN problems (interference, multi-path fading, and accurate energy consumption).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgment

This work was supported in part by the PHC TASSILI 21MDU32.

References

[1] E. Borgia, The Internet of Things vision: Key features, applications and open issues, Comput. Commun. 54 (2014) 1–31, http://dx.doi.org/10.1016/j.comcom.2014.09.008, URL https://www.sciencedirect.com/science/article/pii/S0140366414003168.
[2] L. Atzori, A. Iera, G. Morabito, The Internet of Things: A survey, Comput. Netw. 54 (15) (2010) 2787–2805, http://dx.doi.org/10.1016/j.comnet.2010.05.010, URL https://www.sciencedirect.com/science/article/pii/S1389128610001568.
[3] D. De Guglielmo, S. Brienza, G. Anastasi, IEEE 802.15.4e: A survey, Comput. Commun. 88 (2016) 1–24, http://dx.doi.org/10.1016/j.comcom.2016.05.004, URL https://www.sciencedirect.com/science/article/pii/S0140366416301980.
[4] I. std., 802.15.4e, Part. 15.4: Low-Rate Wireless Personal Area Networks (LR-WPANs) Amendament 1: MAC sublayer, Standard, IEEE standard for Information Technology, 2012, pp. 1–225, http://dx.doi.org/10.1109/IEEESTD.2012.6185525.
[5] X. Vilajosana, K. Pister, T. Watteyne, Minimal IPv6 over the TSCH Mode of IEEE 802.15.4e (6TiSCH) Configuration, RFC 8180, RFC Editor, 2017, http://dx.doi.org/10.17487/RFC8180, URL https://www.rfc-editor.org/info/rfc8180.
[6] A.R. Urke, O. Kure, K. Ovsthus, A survey of 802.15.4 TSCH schedulers for a standardized industrial Internet of Things, Sensors 22 (1) (2022) http://dx.doi.org/10.3390/s22010015, URL https://www.mdpi.com/1424-8220/22/1/15.
[7] M. Maimour, C. Pham, D. Hoang, A congestion control framework for handling video surveillance traffics on WSN, in: 2009 International Conference on Computational Science and Engineering, Vol. 2, 2009, pp. 943–948, http://dx.doi.org/10.1109/CSE.2009.200.
[8] A. Aliouat, N. Kouadria, M. Maimour, S. Harize, N. Doghmane, Region-of-interest based video coding strategy for rate/energy-constrained smart surveillance systems using WMSNs, Ad Hoc Netw. 140 (2023) 103076, http://dx.doi.org/10.1016/j.adhoc.2022.103076, URL https://www.sciencedirect.com/science/article/pii/S1570870522002487.
[9] Y. He, J. Guo, X. Zheng, From surveillance to digital twin: Challenges and recent advances of signal processing for industrial Internet of Things, IEEE Signal Process. Mag. 35 (5) (2018) 120–129.
[10] F. Righetti, C. Vallati, S.K. Das, G. Anastasi, An evaluation of the 6TiSCH distributed resource management mode, ACM Trans. Internet Things 1 (4) (2020) http://dx.doi.org/10.1145/3395927.
[11] A. Elsts, S. Kim, H.-S. Kim, C. Kim, An empirical survey of autonomous scheduling methods for TSCH, IEEE Access 8 (2020) 67147–67165, http://dx.doi.org/10.1109/ACCESS.2020.2980119.
[12] C. Orfanidis, A. Elsts, P. Pop, X. Fafoutis, TSCH evaluation under heterogeneous mobile scenarios, IoT 2 (4) (2021) 656–668, http://dx.doi.org/10.3390/iot2040033, URL https://www.mdpi.com/2624-831X/2/4/33.
[13] D.D. Guglielmo, G. Anastasi, A. Seghetti, From IEEE 802.15.4 to IEEE 802.15.4e: A step towards the internet of things, Adv. Intell. Syst. Comput. (2014).
[14] S. Duquennoy, A. Elsts, B.A. Nahas, G. Oikonomo, TSCH and 6TiSCH for contiki: Challenges, design and evaluation, in: 2017 13th International Conference on Distributed Computing in Sensor Systems, DCOSS, 2017, pp. 11–18, http://dx.doi.org/10.1109/DCOSS.2017.29.
[15] C. Adjih, E. Baccelli, E. Fleury, G. Harter, N. Mitton, T. Noel, R. Pissard-Gibollet, F. Saint-Marcel, G. Schreiner, J. Vandaele, T. Watteyne, FIT IoT-LAB: A large scale open experimental IoT testbed, 2015, Milan, Italy URL https://hal.inria.fr/hal-01213938.
[16] A. Saifullah, Y. Xu, C. Lu, Y. Chen, Real-time scheduling for wirelessHART networks, in: 2010 31st IEEE Real-Time Systems Symposium, 2010, pp. 150–159, http://dx.doi.org/10.1109/RTSS.2010.41.
[17] S. Duquennoy, B.A. Nahas, O. Landsiedel, T. Watteyne, Orchestra: Robust mesh networks through autonomously scheduled TSCH, in: Proceedings of the 13th ACM Conference on Embedded Networked Sensor Systems, 2015.
[18] P. Thubert, T. Winter, RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks, draft-pthubert-roll-rfc6550bis-01, Internet Engineering Task Force, 2021, Work in Progress URL https://datatracker.ietf.org/doc/html/draft-pthubert-roll-rfc6550bis-01.
[19] T. Chang, M. Vučinić, X. Vilajosana, S. Duquennoy, D.R. Dujovne, 6TiSCH Minimal Scheduling Function (MSF), RFC 9033, RFC Editor, 2021, http://dx.doi.org/10.17487/RFC9033, URL https://www.rfc-editor.org/info/rfc9033.
[20] F. Orozco-Santos, V. Sempere-Payá, J. Silvestre-Blanes, T. Albero-Albero, TSCH multiflow scheduling with QoS guarantees: A comparison of SDN with common schedulers, Appl. Sci. 12 (2021) 119, http://dx.doi.org/10.3390/app12010119.
[21] F. Orozco-Santos, V. Sempere-Payá, T. Albero-Albero, J. Silvestre-Blanes, Enhancing SDN WISE with Slicing over TSCH, Sensors 21 (4) (2021) http://dx.doi.org/10.3390/s21041075, URL https://www.mdpi.com/1424-8220/21/4/1075.
[22] A.-C. Anadiotis, L. Galluccio, S. Milardo, G. Morabito, S. Palazzo, SD-WISE: A software-defined wireless sensor network, Comput. Netw. 159 (2019) 84–95, http://dx.doi.org/10.1016/j.comnet.2019.04.029, URL https://www.sciencedirect.com/science/article/pii/S1389128618312192.
[23] R. Hermeto, A. Gallais, F. Theoleyre, Scheduling for IEEE802.15.4-TSCH and slow channel hopping MAC in low power industrial wireless networks: A survey, Comput. Commun. 114 (2017) http://dx.doi.org/10.1016/j.comcom.2017.10.004.
[24] S. Kharb, A. Singhrova, A survey on network formation and scheduling algorithms for time slotted channel hopping in industrial networks, J. Netw. Comput. Appl. 126 (2019) 59–87, http://dx.doi.org/10.1016/j.jnca.2018.11.004, URL https://www.sciencedirect.com/science/article/pii/S1084804518303631.

[25] S. Hammoudi, A. Bentaleb, S. Harous, Z. Aliouat, Scheduling in IEEE 802.15.4e time slotted channel hopping: A survey, in: 2020 11th IEEE Annual Ubiquitous Computing, Electronics Mobile Communication Conference, UEMCON, 2020, pp. 0331–0336, http://dx.doi.org/10.1109/UEMCON51285.2020.9298043.
[26] R.S. Sutton, A.G. Barto, Reinforcement Learning: An Introduction, MIT Press, 2018.
[27] A. Dunkels, The contikimac radio duty cycling protocol, 2012.
[28] A. Dunkels, B. Gronvall, T. Voigt, Contiki - a lightweight and flexible operating system for tiny networked sensors, in: 29th Annual IEEE International Conference on Local Computer Networks, 2004, pp. 455–462, http://dx.doi.org/10.1109/LCN.2004.38.
[29] S. Kim, H.-S. Kim, C. Kim, ALICE: Autonomous link-based cell scheduling for TSCH, in: 2019 18th ACM/IEEE International Conference on Information Processing in Sensor Networks, IPSN, 2019, pp. 121–132, http://dx.doi.org/10.1145/3302506.3310394.
[30] S. Jeong, H.-S. Kim, J. Paek, S. Bahk, OST: On-demand TSCH scheduling with traffic-awareness, in: IEEE INFOCOM 2020 - IEEE Conference on Computer Communications, 2020, pp. 69–78, http://dx.doi.org/10.1109/INFOCOM41043.2020.9155496.
[31] T. Chang, M. Vučinić, X. Vilajosana, D. Dujovne, T. Watteyne, 6TiSCH minimal scheduling function: Performance evaluation, Internet Technol. Lett. 3 (2020) http://dx.doi.org/10.1002/itl2.170.
[32] M.R. Palattella, T. Watteyne, Q. Wang, K. Muraoka, N. Accettura, D. Dujovne, L.A. Grieco, T. Engel, On-the-fly bandwidth reservation for 6TiSCH wireless industrial networks, IEEE Sens. J. 16 (2) (2016) 550–560, http://dx.doi.org/10.1109/JSEN.2015.2480886.
[33] N. Accettura, M.R. Palattella, G. Boggia, L.A. Grieco, M. Dohler, Decentralized traffic aware scheduling for multi-hop low power lossy networks in the Internet of Things, in: 2013 IEEE 14th International Symposium on "a World of Wireless, Mobile and Multimedia Networks", WoWMoM, 2013, pp. 1–6, http://dx.doi.org/10.1109/WoWMoM.2013.6583485.
[34] N. Accettura, E. Vogli, M.R. Palattella, L.A. Grieco, G. Boggia, M. Dohler, Decentralized traffic aware scheduling in 6TiSCH networks: Design and experimental evaluation, IEEE Internet Things J. 2 (6) (2015) 455–470, http://dx.doi.org/10.1109/JIOT.2015.2476915.
[35] M.R. Palattella, N. Accettura, M. Dohler, L.A. Grieco, G. Boggia, Traffic aware scheduling algorithm for reliable low-power multi-hop IEEE 802.15.4e networks, in: 2012 IEEE 23rd International Symposium on Personal, Indoor and Mobile Radio Communications, PIMRC, 2012, pp. 327–332, http://dx.doi.org/10.1109/PIMRC.2012.6362805.
[36] R. Soua, P. Minet, E. Livolant, A distributed joint channel and slot assignment for convergecast in wireless sensor networks, in: 2014 6th International Conference on New Technologies, Mobility and Security, NTMS, 2014, pp. 1–5, http://dx.doi.org/10.1109/NTMS.2014.6813995.
[37] R. Soua, P. Minet, E. Livolant, DiSCA: A distributed scheduling for convergecast in multichannel wireless sensor networks, in: 2015 IFIP/IEEE International Symposium on Integrated Network Management, IM, 2015, pp. 156–164, http://dx.doi.org/10.1109/INM.2015.7140288.
[38] M. Kherbache, O. Sobirov, M. Maimour, E. Rondeau, Reinforcement Learning TDMA-Based MAC Scheduling in Industrial Internet of Things: A Survey, University of Lorraine, 2022.
[39] H. Nguyen-Duy, T. Ngo-Quynh, F. KOJIMA, T. Pham-Van, T. Nguyen-Duc, S. Luongoudon, RL-TSCH: A reinforcement learning algorithm for radio scheduling in TSCH 802.15.4e, in: 2019 International Conference on Information and Communication Technology Convergence, ICTC, 2019, pp. 227–231, http://dx.doi.org/10.1109/ICTC46691.2019.8939833.
[40] C. Watkins, P. Dayan, Technical note: Q-learning, Mach. Learn. 8 (1992) 279–292, http://dx.doi.org/10.1007/BF00992698.
[41] H. Park, H. Kim, S.-T. Kim, P. Mah, Multi-agent reinforcement-learning-based time-slotted channel hopping medium access control scheduling scheme, IEEE Access 8 (2020) 139727–139736, http://dx.doi.org/10.1109/ACCESS.2020.3010575.
[42] K.-H. Phung, T.T. Huong, D. Khanh Dung, V.X. Tuong, T. Pham, T. Nguyen, K. Steenhaut, A scheduler for time slotted channel hopping networks supporting qos differentiated services, in: 2018 International Conference on Advanced Technologies for Communications, ATC, 2018, pp. 232–236, http://dx.doi.org/10.1109/ATC.2018.8587569.
[43] F. Osterlind, A. Dunkels, J. Eriksson, N. Finne, T. Voigt, Cross-level sensor network simulation with COOJA, in: Proceedings. 2006 31st IEEE Conference on Local Computer Networks, 2006, pp. 641–648, http://dx.doi.org/10.1109/LCN.2006.322172.
[44] G. Oikonomou, S. Duquennoy, A. Elsts, J. Eriksson, Y. Tanaka, N. Tsiftes, The contiki-NG open source operating system for next generation IoT devices, SoftwareX 18 (2022) 101089, http://dx.doi.org/10.1016/j.softx.2022.101089.
[45] Tmote Sky : Datasheet, Tech. rep., Moteiv Corporation, 2006, pp. 1–28.